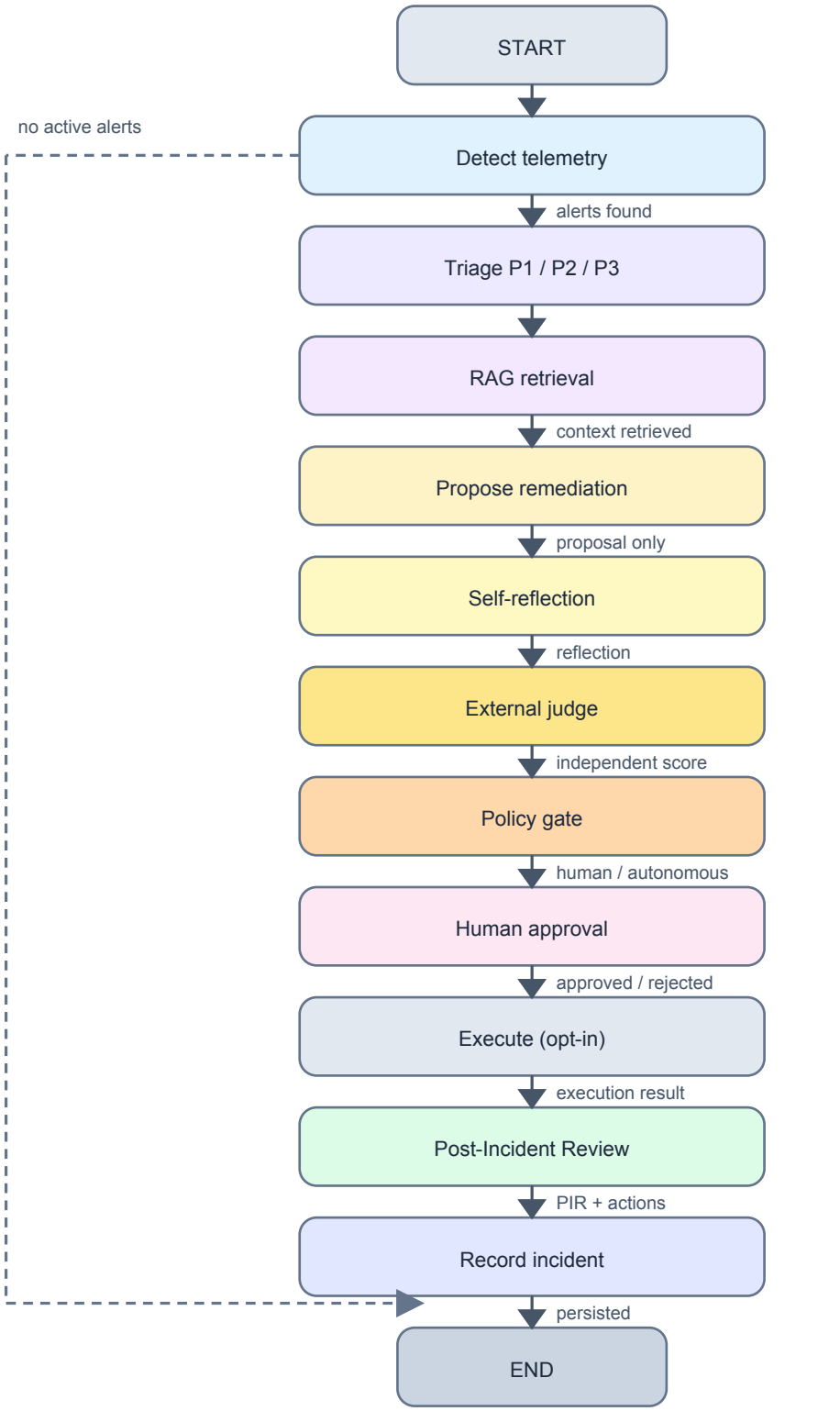
Taller 03 - Incident Commander

LangGraph + MCP telemetry + RAG + human approval + Post-Incident Review

Simulation: eight active Kubernetes alerts from the simulated telemetry MCP.
Safety boundary: kubectl commands are generated and documented, but never executed by this deliverable.
LLM/embedding design: H200 connection in the project; deterministic simulation doubles used for this PDF.

1. LangGraph architecture

The graph is obtained from the compiled LangGraph application, not drawn manually. The conditional path ends when no firing alert exists; otherwise it proceeds through triage, RAG proposal, approval, PIR and memory.



Solid arrows: incident workflow | Dashed arrows: conditional no-alert route

2. Incident detection and triage

The telemetry adapter returned three firing alerts. The classifier prioritizes P1 before P2 and P3 and selects the P1 incident for immediate handling.

Incident ID	Description	Priority	Rank	Agent score	Namespace / service
k8s-prod-api-001	HTTP 5xx responses above the critical threshold for 10 minutes.	P1	1	98	payments / checkout-api

Incident ID	Description	Priority	Rank	Agent score	Namespace / service
k8s-prod-api-005	Login failures are affecting multiple production clients.	P1	2	94	identity / auth-api
k8s-prod-api-006	The dispatch API has no healthy ready endpoints.	P1	3	91	shipping / dispatch-api
k8s-prod-db-002	Consumer lag above 5,000 messages and increasing steadily.	P2	4	82	orders / orders-worker
k8s-prod-db-005	Invoice jobs are delayed beyond the operational objective.	P2	5	76	billing / invoice-worker
k8s-prod-cache-006	Cache evictions are increasing and may degrade catalog latency.	P2	6	68	catalog / redis
k8s-staging-web-003	Intermittent CrashLoopBackOff events in the deployment.	P3	7	52	web / frontend
k8s-staging-web-007	Non-production preview pods have intermittent restarts.	P3	8	32	docs / documentation

Selected incident: k8s-prod-api-001. The triage model reviewed all 8 alerts, compared the three P1 candidates, and assigned the selected alert an impact score of 98 from its evidence. The MCP did not provide this score; it was produced by the triage agent and preserved in the graph state.

Selection rule: priority is considered first; impact score and rationale resolve candidates within the same priority. The complete ranked assessment is preserved in triage.model_assessment.

3. RAG remediation proposal

The remediation agent receives the selected alert, triage result and retrieved Kubernetes knowledge. It proposes commands as text only; no shell, kubectl client or Kubernetes API is called.

Generated proposal:
Propuesta de remediacion para reducir errores 5xx y validar el rollout. No se ejecuta automaticamente.

```
kubectl -n payments rollout restart deployment/checkout-api
kubectl -n payments rollout status deployment/checkout-api --timeout=120s
kubectl -n payments get pods -l app=checkout-api -o wide
kubectl -n payments get events --sort-by=.lastTimestamp | tail -20
```

Retrieved sources:


```
[
{
"chunk_id": "kubernetes_knowledge-p1-c1",
"documento": "kubernetes_knowledge.pdf",
"pagina": 1,
"texto": "Deployments manage stateless Pods; rollout status verifies availability."
},
{
"chunk_id": "arquitectura-kubernetes-actual-p1-c0",
"documento": "arquitectura-kubernetes-actual.pdf",
"pagina": 1,
"texto": "The observed cluster exposes workloads through Kubernetes Services."
}
]
```


4. Human-in-the-loop approval

The graph stopped at interrupt before any command could be applied. The approval payload contained the incident, priority, alert evidence, proposed remediation and the explicit safety message.


```
{
"type": "human_approval_required",
"incident_id": "incident-k8s-prod-api-001",
"priority": "P1",
"alert": {
"id": "k8s-prod-api-001",
"severity": "P1",
"status": "firing",
"cluster": "prod-us-east-1",
"namespace": "payments",
"service": "checkout-api",
"summary": "Checkout API error rate above 20%",
"description": "HTTP 5xx responses above the critical threshold for 10 minutes.",
```

```
"started_at": "2026-10-04T09:42:00Z",
"runbook_url": "https://runbooks.example.com/payments/checkout-api-errors",
"labels": {
"team": "payments",
"tier": "critical"
}
},
"remediation": {
"status": "proposed_not_executed",
"question": "Incidente P1 en Kubernetes: Checkout API error rate above 20%. Namespace=payments, service=checkout-api. Usa la arquitectura y knowledge base recuperadas para proponer una remediacion. Devuelve pasos, YAML o kubectl como codigo, riesgos y una causa probable. No ejecutes nada.",
"proposal": "Propuesta de remediacion para reducir errores 5xx y validar el rollout. No se ejecuta automaticamente.\n\nkubectl -n payments rollout restart deployment/checkout-api\nkubectl -n payments rollout status deployment/checkout-api --timeout=120s\nkubectl -n payments get pods -l app=checkout-api -o wide\nkubectl -n payments get events --sort-by=.lastTimestamp | tail -20",
"contexts": [
{
"chunk_id": "kubernetes_knowledge-p1-c1",
"documento": "kubernetes_knowledge.pdf",
"pagina": 1,
"texto": "Deployments manage stateless Pods; rollout status verifies availability."
},
{
"chunk_id": "arquitectura-kubernetes-actual-p1-c0",
"documento": "arquitectura-kubernetes-actual.pdf",
"pagina": 1,
"texto": "The observed cluster exposes workloads through Kubernetes Services."
}
],
"model": "H200-RAG-simulation"
},
"message": "Aprueba explicitamente la propuesta. Este grafo no ejecuta codigo automaticamente."
}
```


Simulation decision: APPROVED by the human reviewer. This approval authorizes only the simulated validation documented below; the graph itself still does not execute commands.

5. Simulated command application and validation

```
kubectl -n payments rollout restart deployment/checkout-api
kubectl -n payments rollout status deployment/checkout-api --timeout=120s
kubectl -n payments get pods -l app=checkout-api -o wide
kubectl -n payments get events --sort-by=.lastTimestamp | tail -20
```

Check	Simulated result	Conclusion
rollout status	deployment/checkout-api successfully rolled out	PASS
pod health	desired replicas available and Ready	PASS
telemetry	HTTP 5xx rate below critical threshold	PASS
events	no new critical warning after rollout	PASS

Conclusion: the simulated remediation fixed the incident. This is a scenario result, not evidence from a real Kubernetes cluster.

6. Post-Incident Review

Executive summary

A sustained HTTP 5xx rate affected checkout-api in production. The Incident Commander classified the event as P1, retrieved Kubernetes remediation guidance, obtained human approval, and simulated a successful rollout restart and validation.

Priority	Impact	Detection
P1	Checkout requests were degraded for approximately four minutes in the simulation. No data loss was observed. Customer impact and exact production error volume require confirmation from live telemetry.	The telemetry MCP detected HTTP 5xx responses above 20% for ten minutes and raised k8s-prod-api-001.

Incident timeline

Time	Event	Evidence
09:42 UTC	Telemetry detected HTTP 5xx above 20% for checkout-api.	k8s-prod-api-001
09:43 UTC	Classifier assigned P1 and routed the incident.	triage node
09:44 UTC	RAG retrieved Kubernetes architecture guidance and proposed a rollout restart.	retrieve_rag / propose_remediation
09:45 UTC	Human approved the proposed commands for simulation.	approval interrupt
09:46 UTC	Simulated validation reported the error rate below threshold.	simulated checks

Response and resolution

The classifier prioritized the alert, RAG supplied a remediation proposal, and the human approval gate prevented unreviewed execution. Simulate a rollout restart for checkout-api, verify deployment readiness and inspect recent events. All simulated checks passed.

Root cause analysis: Five Whys

The precise production root cause is unconfirmed. The available evidence supports a probable application or dependency failure producing sustained HTTP 5xx responses. Root cause: Probable checkout-api application or dependency failure; confidence is medium until production logs, traces and dependency metrics are reviewed. Confidence: medium.

Why	Finding	Evidence
1	The service returned HTTP 5xx responses above the critical threshold.	P1 telemetry alert
2	A probable application or upstream dependency failure affected request handling.	Hypothesis; logs were not available in the simulation
3	The alert payload did not include dependency-level traces or error fingerprints.	Telemetry MCP payload
4	The available runbook and architecture context supported rollout recovery but not definitive diagnosis.	RAG contexts
5	A production investigation with logs, traces and dependency metrics is required.	Evidence gap recorded by the incident commander

Contributing factors

Insufficient evidence in the simulated telemetry payload; Critical alert required a faster runbook-driven response

Lessons learned

P1 routing and human approval worked as designed; The RAG proposal was useful for safe recovery steps; The telemetry payload should include dependency evidence

7. Incident record and preventive actions

Persistent memory: /Users/AndyEdu/Documents/MAESTRIA IA/IA Generativa y Agentes/Semana3/taller03/output_data/incident_memory.sqlite3
The record_incident node wrote the incident review and preventive action register to SQLite. This database is outside the temporary simulation context and remains available for future incidents.

ID	Action	Owner	Priority	Due	Status
PA-001	Add a runbook link and dashboard panel to the P1 alert.	payments team	P1	2026-10-11	planned
PA-002	Review checkout-api dependency timeouts and error budgets.	payments team	P1	2026-10-18	planned
PA-003	Create a canary validation for future rollouts.	platform team	P2	2026-10-25	planned

8. Trace and state evidence

The final state contains 11 trace events. Each event records node, timestamp, latency and token usage where the underlying H200 call provides it. The approved state also includes the incident ID, selected alert, triage, remediation, approval, review and preventive actions.

Node	Input tokens	Output tokens	Latency / measurement
detect_telemetry	N/A	N/A	0.0001 s | not_applicable_no_llm_call
triage	310	72	0.4200 s | measured
retrieve_rag	N/A	N/A	0.0000 s | not_applicable_no_llm_call
propose_remediation	260	145	0.8600 s | measured
self_reflect	N/A	N/A	0.0004 s | not_applicable_no_llm_call

Node	Input tokens	Output tokens	Latency / measurement
external_judge	N/A	N/A	0.0019 s | not_applicable_no_llm_call
policy_gate	N/A	N/A	0.0008 s | not_applicable_no_llm_call
approve_remediation	N/A	N/A	0.0010 s | not_applicable_no_llm_call
execute_remediation	N/A	N/A	0.0000 s | not_applicable_no_llm_call
post_incident_review	420	210	1.2400 s | measured
record_incident	N/A	N/A	0.0157 s | not_applicable_no_llm_call

9. Safety and limitations

The deliverable intentionally stops at approval. A production executor would need an allowlist of commands, Kubernetes identity and RBAC, dry-run support, namespace restrictions, timeouts, audit logging and a second confirmation for destructive actions. The simulated PASS results must not be interpreted as live cluster evidence.

10. Evidence of agents, LangGraph, RAG and tools

The implementation uses specialized agent roles represented as LangGraph nodes. They cooperate through the shared IncidentState and leave trace events; they are not seven independent processes.

Agent	Role	Implementation evidence
A1 Telemetry observer	Detect active incidents and select candidates	detect_telemetry + LocalTelemetryClient
A2 Triage classifier	Review alert evidence, score impact and select among P1/P2/P3	triage_incident + H200 JSON assessment
A3 RAG investigator	Retrieve architecture and Kubernetes knowledge	retrieve_rag + RagTaller03 + Qdrant
A4 Remediation engineer	Propose kubectl/YAML remediation without execution	propose_remediation + H200.chat
A5 Safety reviewer	Pause and request human approval	approve_remediation + interrupt
A6 PIR analyst	Produce timeline, RCA and five whys	post_incident_review + H200 JSON mode
A7 Learning curator	Persist incidents and preventive actions	record_incident + IncidentMemory SQLite

LangGraph evidence

StateGraph: IncidentState, START, END and conditional routing after detect_telemetry.
Human-in-the-loop: interrupt() pauses the graph; Command(resume=...) continues it.
Persistence: InMemorySaver checkpoints the running thread; IncidentMemory persists completed incidents in SQLite.
Trace: every node appends _traza with timestamp, latency, token status and node-specific details.

RAG evidence

Ingestion: the Kubernetes architecture and official knowledge PDFs are fragmented.
Embedding: H200 bge-m3 creates vectors.
Vector database: Qdrant collection taller03_kubernetes stores vectors and chunk payloads.
Retrieval: retrieve_rag calls RagTaller03.recuperar() and passes the selected contexts to propose_remediation.
Generation: H200.chat() receives only the alert, triage and retrieved context before proposing remediation.

MCP tool evidence

Tool	Purpose	Evidence in telemetry_mcp.py
list_alerts	List and filter firing/resolved P1/P2/P3 alerts	@mcp.tool()
get_alert	Retrieve one alert by ID	@mcp.tool()
summarize_alerts	Count alerts grouped by severity	@mcp.tool()

Minimum achieved: 7 specialized agent roles, LangGraph orchestration, one RAG path backed by H200 and Qdrant, and 3 MCP telemetry tools.